GitHub Pages

https://docs.github.com/de/pages/getting-started-with-github-pages/creating-a-github-pages-site
https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site

Variante 1:

Zuerst je nach dem ob im root (/) oder im /docs ordner die gewünschte HTML Page erstellen. Alles muss statisch sein.

Repository erstellen > Settings > Pages

Dort gibt es Source, wo man "Deploy from a branch" oder "GitHub Actions". In Variante 1 nehmen wir "Deploy from a branch" beim Source.

Dannach bei branch kann man den spezifischen branch angeben, von wo er diese holen soll und zudem ob er es im "root (/)" oder im "/docs" ordner diese statische seite findet.

Dort kann man nur statische Sachen einfügen, also HTML, CSS und Javascript.
"Standardgemäss wird es mit Jekyll erstellt"

Schritt 1: GitHub Pages auf "Deploy from a branch" stellen
Schritt 2: Grunddateien anlegen (Statische Website/ HTML in root oder /docs file reinmachen)

Variante 2:

Diesesmal mit GitHub Actions.

Anstatt HTML pages nehmen wir hier markdown pages (.md). Diese werden mit MKDocs oder anderen Tools automatisch in HTML pages generiert. Anschliesslich mit dem Workflow automatisch deployed/ publiziert.

Wir wählen nun beim Source GitHub Actions" aus. Man kann standardgemäss schon einen vordefinierten Workflow von GitHub nehmen -> z.B. GitHub Pages Jekyll, aber wir nehmen mal MKDocs. Es macht das

Diesesmal muss man einen workflow erstellen

Schritt 1: GitHub Pages auf Action stellen
Schritt 2: Grunddateien anlegen (Die .md dateien im /docs ordner und das mkdocs.yml im root)
Schritt 3: Workflow erstellen (im .github/workflows/pages.yml Inhalt eingeben vom Workflow)

Optional: Lokales testen in Git Bash

```
pip install mkdocs
mkdocs serve
```

```
mkdocs build
```

Machen MKDocs und GitHub Pages mit Jekyll grob dasselbe?

Grob ja:
Beide sind **Static Site Generators**. Beide nehmen Quelldateien wie Markdown plus Konfiguration und erzeugen daraus **statische HTML-Seiten**. GitHub Pages kann Jekyll direkt verwenden, und MkDocs baut ebenfalls komplett statische HTML-Sites.

Der grobe Unterschied

Jekyll

- allgemeiner Static Site Generator
- stark in Blog-/Website-Logik
- Ruby-basiert
- historisch eng mit GitHub Pages verbunden
- GitHub Pages baut bei Branch-Publishing standardmässig mit Jekyll, sofern du das nicht deaktivierst oder stattdessen Actions verwendest.

MKDocs

- speziell für **Dokumentation**
- Python-basiert
- sehr einfacher Einstieg für Markdown-Dokus
- klare Doku-Struktur mit `docs/` + `mkdocs.yml`
- erzeugt ebenfalls statisches HTML, aber mit Fokus auf technische Dokumentation, Navigation und Doku-Layouts.

Praktisch gedacht

Für deine Präsentation kannst du es so merken:

- **Jekyll** = eher allgemeiner Website-/Blog-Generator
- **MKDocs** = eher Doku-/Projektwiki-Generator

Wichtig auf GitHub Pages

Wenn du **„Deploy from a branch“** verwendest, baut GitHub Pages standardmässig mit **Jekyll**.
Wenn du aber **MKDocs** willst, ist der saubere Weg **GitHub Actions**, also selbst bauen und danach das HTML deployen. GitHub dokumentiert genau diesen Fall.

Workflow "Static HTML" bei GitHub Actions vs Deploy from a branch

Beides ist praktisch fast das selbe. Beide sind bereits statisch, also in HTML und zudem muss nichts generiert werden. **Für fertige statische HTML/CSS/JS-Dateien ist das Endergebnis bei beiden fast gleich**: Am Schluss liegt eine statische Website auf GitHub Pages. Der Unterschied ist also **nicht primär im Resultat**, sondern **im Weg dorthin und in der Kontrolle darüber**. GitHub sagt selbst: Wenn du **keine Kontrolle über den Build-Prozess brauchst**, ist **„Deploy from a branch“** die empfohlene einfachere Variante.

Der praktische Unterschied

1. Deploy from a branch

Das ist die **einfachste Variante**:

- GitHub veröffentlicht direkt den Inhalt eines Branches
- als Quelle geht nur:
  - Root `/`
  - oder `/docs`
- du brauchst **keinen Workflow**
- bei Änderungen im Publishing-Branch wird automatisch neu veröffentlicht

Für dein Beispiel heisst das:

- du hast `index.html`, `style.css`, `script.js`
- pushst auf `main`
- stellst Pages auf `main` + `/docs` oder `/`
- fertig

Das ist ideal, wenn du einfach nur eine statische Demo zeigen willst.

---

2. GitHub Actions

Hier steuerst du den Publikationsprozess selbst:

- wann deployed wird
- welche Schritte vorher laufen
- ob Tests / Checks / Build / Minify / Copy / Umstrukturierung passieren
- welches Verzeichnis am Ende als Artifact veröffentlicht wird

Auch wenn du **nichts generieren** musst, kann Actions trotzdem sinnvoll sein, wenn du z. B.:

- nur nach erfolgreichem Test deployen willst
- vor dem Deploy Dateien prüfen willst
- HTML aus einem Unterordner kopieren willst
- mehrere Schritte kombinieren willst
- ein sauber kontrolliertes CI/CD-Setup zeigen willst

Der Kernunterschied in einem Satz

- **Deploy from a branch** = „Nimm vorhandene Dateien direkt aus dem Repo und veröffentliche sie.“
- **GitHub Actions** = „Führe zuerst meinen definierten Workflow aus und veröffentliche dann das Ergebnis.“